COMPUTER VISION

Theory, Mathematical Foundations & Function Reference

Exam preparation • Technical interviews • Practical OpenCV revision

Prepared from the uploaded practice notebooks

Topics covered: image representation, colour channels, transformations, drawing, arithmetic and bitwise operations, smoothing, sharpening, thresholding, edge detection, contours, webcam video and Haar-cascade detection.

Study edition | November 2026

How to use these notes

This document follows the techniques visible in the supplied notebooks. Each topic explains the concept, why it is used, the relevant mathematics, important OpenCV/Python functions, and likely exam or interview points. The code examples are compact reference patterns rather than a replacement for running the original notebooks.

Scope note: The uploaded notebooks demonstrate classical image processing and basic object detection with Haar cascades. They do not demonstrate CNN training, YOLO, image segmentation networks, feature descriptors such as SIFT/ORB, or deep-learning inference. Those topics are intentionally not presented as completed practicals.

Contents

• 1. Computer vision and digital image fundamentals

• 2. Reading, displaying, saving and inspecting images

• 3. Colour spaces and channel manipulation

• 4. Geometric transformations and drawing

• 5. Image arithmetic and bitwise operations

• 6. Smoothing and noise reduction: Gaussian and median blur

• 7. Image sharpening and spatial filtering

• 8. Thresholding and binary images

• 9. Canny edge detection

• 10. Contours, shape approximation and hierarchy

• 11. Webcam capture and video processing

• 12. Haar-cascade face detection

• 13. Function quick reference, common errors and interview questions

1. Computer vision and digital image fundamentals

Computer vision is the field of computing that enables machines to obtain useful information from images and video. A typical pipeline is: acquire image → preprocess → extract useful structures/features → detect or classify → interpret the result.

An image is a two-dimensional grid of pixels. A grayscale image stores one intensity value per pixel; a colour image commonly stores three channel values per pixel.

1.1 Pixel and image representation

For an image of height H, width W and C channels, its array shape is commonly (H, W, C). A grayscale image usually has shape (H, W). In an 8-bit image, each channel typically ranges from 0 to 255.

For a grayscale image, I(x,y) is the intensity at coordinate (x,y). For a colour image, the pixel is a vector such as [B,G,R] in OpenCV's default channel order.

Storage estimate for an uncompressed uint8 image: H × W × C bytes. For example, a 500 × 500 RGB image requires about 750,000 bytes (not including metadata or array overhead).

1.2 Coordinate system

• The origin (0,0) is at the top-left corner.

• x increases from left to right; y increases from top to bottom.

• A point is usually written (x,y), while NumPy indexing is image[y,x].

• For a rectangle, OpenCV commonly uses top-left (x,y) and bottom-right (x+w,y+h).

2. Reading, displaying, saving and inspecting images

2.1 OpenCV image I/O

```
import cv2

img = cv2.imread("photo.jpg")                # BGR colour by default
gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:
    raise ValueError("Image could not be loaded")

cv2.imshow("Image", img)
cv2.waitKey(0)                               # wait for a key
cv2.destroyAllWindows()

cv2.imwrite("output.png", img)
```

Function / property	Purpose / important behaviour
cv2.imread(path, flag)	Reads an image. Returns None if the path cannot be read.
cv2.IMREAD_COLOR	Loads a colour image (default; normally 3 channels, BGR).
cv2.IMREAD_GRAYSCALE	Loads a single-channel grayscale image.
cv2.imshow(title, image)	Displays an image in an OpenCV window.
cv2.waitKey(delay)	Waits for a key; delay is milliseconds. 0 means wait indefinitely.
cv2.destroyAllWindows()	Closes OpenCV display windows.
cv2.imwrite(path, image)	Writes an image to disk; check the returned Boolean.
img.shape / img.dtype	Array dimensions and data type.
type(img)	Usually numpy.ndarray for an OpenCV-loaded image.

2.2 Displaying with Matplotlib

OpenCV loads colour images in BGR order, while Matplotlib expects RGB for normal colour display. Convert before plotting, otherwise red and blue appear swapped.

```
import matplotlib.pyplot as plt

rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(rgb)
plt.axis("off")
plt.show()
```

Pillow (PIL) is another image library. The notebooks use Image.open(), np.array(), rotate(), resize(), crop(), transpose(), filter() and split(). Many Pillow transformations return a new image rather than modifying the original object; assign the result.

```
from PIL import Image, ImageFilter
import numpy as np

pil_img = Image.open("photo.jpg")
arr = np.array(pil_img)
rotated = pil_img.rotate(180)
resized = pil_img.resize((500, 300))
cropped = pil_img.crop((left, top, right, bottom))
flipped = pil_img.transpose(Image.FLIP_LEFT_RIGHT)
blurred = pil_img.filter(ImageFilter.BLUR)
red, green, blue = pil_img.split()
```

3. Colour spaces and channel manipulation

A colour model describes how colours are represented numerically. In an RGB image, each pixel has red, green and blue components. OpenCV's usual decoded colour order is BGR, so channel index 0 is blue, 1 is green and 2 is red.

3.1 Grayscale conversion

A common luminance approximation is:

Y = 0.299R + 0.587G + 0.114B

This weighted sum reflects the human visual system's greater sensitivity to green. OpenCV performs the conversion with cv2.cvtColor().

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb  = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

3.2 Accessing channels

```
# OpenCV BGR array
blue  = img[:, :, 0]
green = img[:, :, 1]
red   = img[:, :, 2]

# Set a channel to zero or maximum
copy = img.copy()
copy[:, :, 2] = 0       # remove red contribution
copy[:, :, 1] = 255     # set green channel to maximum
```

NumPy slicing uses [row, column, channel]. Channel extraction is useful for visualising colour components, colour-based segmentation and understanding image storage.

4. Geometric transformations and drawing

Geometric transformations change the spatial arrangement of pixels. They are used for resizing, orientation correction, augmentation, cropping and preparing inputs for later algorithms.

4.1 Resizing

Resizing maps an input image to a new width and height. If the scale factors are s_x and s_y, then x' = s_x x and y' = s_y y (conceptually; interpolation estimates new pixel values).

```
small = cv2.resize(img, (100, 100))   # (width, height)
half  = cv2.resize(img, None, fx=0.5, fy=0.5,
                   interpolation=cv2.INTER_AREA)
```

INTER_AREA is often suitable for shrinking; INTER_LINEAR is a common general-purpose choice; INTER_CUBIC can be used for enlargement at higher computational cost.

4.2 Flipping

```
vertical   = cv2.flip(img, 0)    # flip around x-axis: top/bottom
horizontal = cv2.flip(img, 1)    # flip around y-axis: left/right
both       = cv2.flip(img, -1)   # both axes
```

4.3 Drawing primitives

Function	Main arguments / purpose
cv2.line(img, p1, p2, color, thickness)	Draws a line between two points.
cv2.rectangle(img, p1, p2, color, thickness)	Draws a rectangle from top-left to bottom-right.
cv2.circle(img, center, radius, color, thickness)	Draws a circle; thickness=-1 fills it.
cv2.polylines(img, pts, isClosed, color, thickness)	Draws one or more connected line sequences.
cv2.putText(img, text, org, font, scale, color, thickness)	Writes text; org is the bottom-left text origin.

```
canvas = np.zeros((500, 500, 3), dtype=np.uint8)
cv2.rectangle(canvas, (50, 50), (200, 180), (0, 255, 0), 2)
cv2.circle(canvas, (250, 250), 40, (255, 0, 0), -1)
```

```
cv2.line(canvas, (0, 0), (499, 499), (255, 255, 255), 2)
cv2.putText(canvas, "CV", (20, 450),
            cv2.FONT_HERSHEY_SIMPLEX, 2, (255,255,255), 2)
```

OpenCV drawing colours are supplied in BGR order. A thickness of -1 fills closed shapes. Use uint8 for ordinary 8-bit image buffers; the notebook's int32 canvas is valid for drawing demonstrations but is not the usual image storage type.

5. Image arithmetic and bitwise operations

Image arithmetic combines corresponding pixels. For element-wise operations, input images generally need compatible dimensions and channel counts.

5.1 Addition and weighted blending

Addition: S(x,y) = I■(x,y) + I■(x,y). OpenCV cv2.add() uses saturated arithmetic for common uint8 images: values above 255 are clipped to 255.

Weighted blending: dst = αI■ + βI■ + γ. α and β control each image's contribution; γ is a scalar brightness offset.

```
added = cv2.add(img1, img2)
blend = cv2.addWeighted(img1, 0.6, img2, 0.4, 10)
```

When α + β = 1 and γ = 0, the blend is a convex weighted average (assuming non-negative weights). The notebook explores different weights and γ values.

5.2 Subtraction

cv2.subtract(I■,I■) performs element-wise saturated subtraction for unsigned 8-bit images: negative results become 0. It is useful for simple change/difference visualisation, though robust change detection usually needs alignment and thresholding.

5.3 Bitwise operations

Bitwise operations act on the binary representation of each channel value. They are especially useful with binary masks.

Operation	Pixel-wise meaning	OpenCV function
AND	1 only where both corresponding bits are 1	cv2.bitwise_and(a,b)
OR	1 where either corresponding bit is 1	cv2.bitwise_or(a,b)
XOR	1 where corresponding bits differ	cv2.bitwise_xor(a,b)
NOT	Inverts every bit	cv2.bitwise_not(a)

For an 8-bit channel, NOT is equivalent to 255 – value. With a mask argument, OpenCV applies the operation only at non-zero mask locations.

Important correction for interviews: XOR and subtraction are not generally equivalent. They may look similar for particular sample images, but they have different definitions and produce different results in general. Also, cv2.bitwise_not(src, dst) treats the second positional argument as the destination array, not as a second source image.

```
bit_and = cv2.bitwise_and(img1, img2)
bit_or  = cv2.bitwise_or(img1, img2)
bit_xor = cv2.bitwise_xor(img1, img2)
bit_not = cv2.bitwise_not(img1)
```

6. Smoothing and noise reduction

Smoothing (blurring) reduces rapid intensity variations. It can suppress noise and make later operations such as thresholding or edge detection more stable. The trade-off is loss of fine detail and edge sharpness.

6.1 Gaussian blur

Gaussian blur replaces each pixel with a weighted average of nearby pixels. Nearby pixels receive larger weights according to a Gaussian distribution.

2-D Gaussian: $G(x,y) = (1 / (2\pi\sigma^2)) \exp(-(x^2+y^2)/(2\sigma^2))$

The filtered image is the convolution $I' = I * G$. Kernel size must be positive and odd in each dimension for the usual OpenCV call. σ controls the spread of the Gaussian; if zero, OpenCV derives it from the kernel size.

```
blur = cv2.GaussianBlur(img, (7, 7), 2)
```

Use cases: reducing approximately Gaussian noise, pre-processing before edge detection, and reducing small texture variations. Larger kernels or sigma values create stronger blur.

6.2 Median blur

Median filtering replaces a pixel with the median of the values in its local neighbourhood. For a 3×3 window, sort the nine values and select the middle value.

Median: $I'(x,y) = \text{median}\{ I(x+i,y+j) : (i,j) \text{ in neighbourhood } \}$

```
median = cv2.medianBlur(img, 3)
```

Median blur is particularly effective against salt-and-pepper noise (isolated bright and dark pixels) and tends to preserve edges better than a simple averaging filter. The kernel size k must be odd and greater than 1.

Gaussian blur	Median blur
Weighted average of neighbours	Median/order statistic of neighbours
Often useful for Gaussian-like noise	Especially useful for salt-and-pepper noise
Can soften edges	Often preserves edges better

7. Image sharpening and spatial filtering

Sharpening emphasises high-frequency components such as edges and fine details. A spatial filter applies a kernel (small matrix) around each pixel.

For a kernel K, convolution-like filtering is: $O(x,y) = \Sigma\blacksquare\Sigma\blacksquare\ K(i,j)\ I(x-i,y-j)$. OpenCV's filter2D applies a correlation-style kernel operation; symmetric kernels give the same result under correlation and convolution.

The notebook uses this sharpening kernel:

```
kernel = np.array([
    [ 0, -1,  0],
    [-1,  5, -1],
    [ 0, -1,  0]
])
sharp = cv2.filter2D(img, -1, kernel)
```

The centre weight is 5 and the four direct neighbours have weight –1. This boosts the centre relative to surrounding pixels. The kernel coefficients sum to 1, so a constant region is approximately preserved while local transitions are enhanced.

Parameter note: In cv2.filter2D(src, ddepth, kernel), ddepth=-1 means the output uses the same depth as the input. Sharpening can amplify noise and create clipping or halos if applied too strongly.

8. Thresholding and binary images

Thresholding converts a grayscale image into a binary or segmented image by comparing each pixel intensity with a threshold T.

Binary threshold: dst(x,y) = maxVal if src(x,y) > T, otherwise 0.

Binary inverse: dst(x,y) = 0 if src(x,y) > T, otherwise maxVal.

```
gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
ret, binary = cv2.threshold(gray, 120, 255, cv2.THRESH_BINARY)
```

```
ret, inverse = cv2.threshold(gray, 120, 255, cv2.THRESH_BINARY_INV)
```

The returned ret value is the threshold used (T for the fixed threshold modes shown). The second returned value is the thresholded image.

Threshold type	Rule / effect
THRESH_BINARY	Bright side of threshold becomes maxVal.
THRESH_BINARY_INV	Bright side becomes 0; lower side becomes maxVal.
THRESH_TRUNC	Values above T are clipped to T.
THRESH_TOZERO	Values at or below T become 0.
THRESH_TOZERO_INV	Values above T become 0.

Thresholding is useful for separating foreground from background when object and background intensities differ. A fixed global threshold can fail under uneven illumination; adaptive thresholding or Otsu's method can help, but those methods were not implemented in the supplied threshold notebook.

9. Canny edge detection

An edge is a location where image intensity changes sharply. Canny is a multi-stage edge detector designed to detect meaningful edges while suppressing noise and reducing multiple responses.

9.1 Canny stages

• Noise reduction: commonly Gaussian smoothing before gradient calculation.

• Gradient calculation: estimate intensity change in x and y directions.

• Gradient magnitude and direction: identify strong changes and edge orientation.

• Non-maximum suppression: thin broad gradient responses to approximately one-pixel edges.

• Double threshold: classify pixels as strong, weak or non-edge using two thresholds.

• Edge tracking by hysteresis: retain weak pixels connected to strong edges; discard isolated weak responses.

For image I, approximate gradients are Gx and Gy. Gradient magnitude $M = \sqrt{(Gx^2 + Gy^2)}$, and direction θ = atan2(Gy,Gx).

The notebook calls Canny with lower threshold 50 and upper threshold 150:

```
gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(gray, 50, 150)
```

Pixels above the high threshold are strong edges; pixels between thresholds are weak candidates; pixels below the low threshold are rejected. The low/high thresholds affect sensitivity and edge connectivity.

10. Contours, shape approximation and hierarchy

A contour is a curve joining continuous boundary points of an object having the same colour or intensity. In practical OpenCV workflows, contours are commonly extracted from a binary image.

10.1 Contour workflow

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 240, 255,
                          cv2.THRESH_BINARY_INV)

contours, hierarchy = cv2.findContours(
    thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE
)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
```

Function	Meaning
cv2.findContours(image, mode, method)	Finds object boundaries in a binary image.
cv2.RETR_TREE	Retrieves all contours and reconstructs full parent/child hierarchy.
cv2.RETR_EXTERNAL	Retrieves only outermost contours.
cv2.CHAIN_APPROX_SIMPLE	Compresses horizontal, vertical and diagonal segments by keeping key endpoints.
cv2.drawContours(...)	Draws selected contours; contourIdx=-1 means all contours.
cv2.contourArea(c)	Computes contour area.
cv2.arcLength(c, closed)	Computes contour perimeter/arc length.
cv2.approxPolyDP(c, epsilon, closed)	Approximates a contour with fewer vertices.

10.2 Polygon approximation

Douglas–Peucker approximation reduces the number of points while keeping the approximating curve within a tolerance ε of the original contour.

ε **= 0.08 × perimeter** is the setting used in the notebook. The approximation is obtained using cv2.approxPolyDP(contour, epsilon, True). A smaller epsilon preserves more detail; a larger epsilon produces a simpler polygon.

The number of approximated vertices is used as a simple shape heuristic: 3 → triangle, 4 → quadrilateral, 5 → pentagon, 6 → hexagon, 7 → heptagon, 8 → octagon, and more than 8 may be treated as circle-like.

Important limitation: This vertex-count rule is a heuristic, not a robust universal classifier. Noise, perspective, rotation, contour quality and epsilon can change the number of vertices. A four-sided contour is not automatically a rectangle or square; check angles and side lengths if that distinction matters.

10.3 Filtering noise and geometric measurements

```
for contour in contours:
    if cv2.contourArea(contour) < 100:
        continue
    perimeter = cv2.arcLength(contour, True)
    approx = cv2.approxPolyDP(contour, 0.08 * perimeter, True)
    corners = len(approx)
```

The area filter removes tiny contours that may have arisen from noise. The threshold 100 is application- and image-resolution-dependent and should be tuned.

10.4 Contour hierarchy

With RETR_TREE, hierarchy describes relationships between contours. OpenCV stores next contour, previous contour, first child and parent indices. This helps distinguish outer boundaries from holes or nested objects.

11. Webcam capture and video processing

A video is a sequence of frames. A webcam pipeline repeatedly captures a frame, processes or displays it, checks for a quit key, and releases the camera resource.

```
cap = cv2.VideoCapture(0)  # usually default webcam

if not cap.isOpened():
    raise RuntimeError("Could not open webcam")

try:
    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

```
        cv2.imshow("Webcam", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
finally:
    cap.release()
    cv2.destroyAllWindows()
```

Function	Purpose
cv2.VideoCapture(0)	Opens default camera; a filename can be used for a video file.
cap.isOpened()	Checks whether the capture device/source opened.
cap.read()	Returns (success_boolean, frame).
cv2.waitKey(1)	Allows the display window to refresh and checks key input.
cap.release()	Releases camera or video resource.

Use try/finally so the camera is released even if an error occurs. The notebook uses q as the quit key. A common issue is a camera permission or device-index problem.

12. Haar-cascade face detection

The practice notebook loads a pretrained frontal-face Haar cascade XML file, reads webcam frames, converts each frame to grayscale, and detects faces with detectMultiScale(). The XML file stores the learned cascade model; it is not a normal image.

12.1 Main idea

The Viola–Jones family of detectors uses Haar-like rectangular intensity features, integral images for fast feature computation, AdaBoost to select useful features, and a cascade of increasingly selective stages. Early stages reject obvious non-face windows cheaply; later stages examine more promising windows.

A Haar-like feature compares sums of pixel intensities over adjacent rectangular regions. A simple two-rectangle feature can be represented as:

f = sum(white region) – **sum(black region)**

The integral image allows a rectangular sum to be computed from four corner values. For a summed-area table S, a rectangle sum can be calculated as S(D) – S(B) – S(C) + S(A), with corners labelled consistently.

12.2 Cascade detection code

```
face_cascade = cv2.CascadeClassifier(
    "haarcascade_frontalface_default.xml"
)
if face_cascade.empty():
    raise ValueError("Cascade XML did not load")

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(
    gray, scaleFactor=1.1, minNeighbors=5
)

for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x+w, y+h),
                  (0, 255, 0), 2)
```

Parameter / function	Explanation
cv2.CascadeClassifier(path)	Loads a trained cascade classifier from an XML file.
empty()	True if the classifier has not loaded a valid model.
detectMultiScale(image, scaleFactor, minNeighbors)	Scans image windows at multiple scales and returns detected rectangles.

Parameter / function	Explanation
scaleFactor=1.1	Each image scale is reduced by roughly 10% between scans. Smaller values scan more scales and can b
minNeighbors=5	Controls how many neighbouring detections are needed to retain a candidate; higher values usually reduc
(x,y,w,h)	Top-left coordinate, width and height of each detection.

The detector is most reliable for reasonably frontal, visible faces similar to its training examples. Pose, lighting, occlusion, image scale and camera quality affect results. It is a classical detector and is generally less robust than modern deep-learning detectors in difficult conditions.

13. Function quick reference

Area	Key functions used in the notebooks
Image I/O	cv2.imread, cv2.imshow, cv2.waitKey, cv2.destroyAllWindows, cv2.imwrite
Colour / arrays	cv2.cvtColor, cv2.COLOR_BGR2GRAY, cv2.COLOR_BGR2RGB, NumPy slicing, shape, dtype, copy
Pillow	Image.open, rotate, resize, crop, transpose, filter, split
Transform / drawing	cv2.resize, cv2.flip, cv2.line, cv2.rectangle, cv2.circle, cv2.polylines, cv2.putText
Arithmetic / logical	cv2.add, cv2.addWeighted, cv2.subtract, cv2.bitwise_and, cv2.bitwise_or, cv2.bitwise_xor, cv2.bitwise_not
Filtering	cv2.GaussianBlur, cv2.medianBlur, cv2.filter2D
Segmentation / edges	cv2.threshold, cv2.Canny
Contours	cv2.findContours, cv2.drawContours, cv2.contourArea, cv2.arcLength, cv2.approxPolyDP
Video / detection	cv2.VideoCapture, isOpened, read, release, cv2.CascadeClassifier, detectMultiScale

14. Common mistakes and practical checks

• Always check whether cv2.imread() returned None before processing an image.

• Remember OpenCV colour arrays are normally BGR; convert to RGB for Matplotlib display.

• NumPy indexing is [y, x, channel], while drawing points are written (x, y).

• For cv2.resize(), dsize is (width, height), not (height, width).

• Use odd kernel dimensions for Gaussian and median blur; medianBlur requires an odd ksize greater than 1.

• Make sure the two images in add/addWeighted/subtract have compatible sizes and channel counts.

• Do not assume XOR is equivalent to subtraction.

• Use a binary image for contour extraction; inspect the threshold result if contours are missing or excessive.

• Tune contour-area cutoffs and polygon epsilon to the image scale and noise level.

• Check CascadeClassifier.empty() and verify that the XML path is correct.

• Release VideoCapture and close windows when the video loop ends.

15. Interview and college viva questions

Q. What is the difference between image processing and computer vision?
Answer: Image processing transforms or analyses images (for example blur, sharpen, threshold). Computer vision uses image information to infer meaning or make decisions (for example detect a face or recognise a shape). The fields overlap.

Q. Why convert an image to grayscale before Canny or Haar detection?
Answer: It reduces three colour channels to one intensity channel, reducing computation and matching the expected input of these classical algorithms.

Q. What is the difference between Gaussian and median blur?
Answer: Gaussian blur uses a weighted local average and is useful for Gaussian-like noise. Median blur uses the neighbourhood median and is particularly effective for salt-and-pepper noise.

Q. What is a kernel in image processing?
Answer: A small matrix of coefficients moved across an image to compute each output pixel from its neighbourhood.

Q. Why does the sharpening kernel contain negative values?
Answer: Negative neighbour weights subtract surrounding intensity so local contrast and edge transitions are emphasised.

Q. What does thresholding do?
Answer: It compares each pixel intensity with a threshold and assigns output values, often creating a binary foreground/background image.

Q. What is the difference between binary and binary-inverse thresholding?
Answer: They reverse which side of the threshold maps to zero and which maps to maxVal.

Q. What are edges?
Answer: Locations where image intensity changes sharply, often corresponding to object boundaries or texture transitions.

Q. Name the main stages of Canny.
Answer: Noise smoothing, gradient calculation, non-maximum suppression, double thresholding and edge tracking by hysteresis.

Q. What is a contour?
Answer: A curve representing a boundary of connected foreground pixels in a binary image.

Q. Why use contourArea() before polygon approximation?
Answer: To discard tiny contours likely caused by noise and focus on meaningful objects.

Q. What does epsilon control in approxPolyDP()?
Answer: The maximum approximation tolerance; larger epsilon generally produces fewer vertices and a simpler contour.

Q. What is contour hierarchy?
Answer: The parent-child and sibling relationships between contours, useful for nested boundaries and holes.

Q. What does detectMultiScale() mean?
Answer: It scans the image at multiple scales so an object can be detected at different apparent sizes.

Q. What is minNeighbors?
Answer: A grouping/acceptance parameter; increasing it generally makes detections more conservative.

Q. Why use an integral image in Haar detection?
Answer: It makes rectangular region-sum calculations fast, independent of rectangle area after the integral image is built.

Q. What is the purpose of cap.release()?
Answer: It frees the webcam/video capture resource.

Q. Why is BGR versus RGB important?
Answer: The same three numeric values represent different colours depending on channel order; displaying BGR as RGB swaps red and blue.

Q. What is the difference between cv2.add() and the + operator?
Answer: cv2.add() uses OpenCV's saturated arithmetic for common uint8 inputs; NumPy + may wrap around on uint8 overflow.

Q. Why is a Haar cascade considered a classical detector?
Answer: It relies on hand-designed Haar-like features, integral images, boosting and a staged cascade rather than learned deep convolutional feature maps.

16. Short revision sheet

Concept	Remember
Image shape	(height, width, channels); pixel indexing image[y,x].
OpenCV colour	BGR by default; Matplotlib display normally needs RGB.
Grayscale	Y ≈ 0.299R + 0.587G + 0.114B.
Weighted blend	dst = αI■ + βI■ + γ.
Gaussian	Weighted average; Gaussian kernel; σ controls spread.
Median	Median of local neighbourhood; good for salt-and-pepper noise.
Sharpen	Enhances high-frequency detail; may amplify noise.
Threshold	Compare intensity with T to create binary output.
Canny	Gradient + thinning + double threshold + hysteresis.
Contours	Boundaries extracted from binary foreground regions.
Polygon approximation	Douglas–Peucker; epsilon controls simplification.
Haar cascade	Haar-like features + integral image + AdaBoost + cascade.
Webcam loop	read frame → process/display → key check → release.

Final study advice: For each practical, be ready to explain (1) the input and output, (2) why the function is used, (3) its important parameters, (4) the mathematical idea, (5) one real-world use, and (6) one limitation. This structure works well for both viva answers and technical interviews.

APPENDIX: COMPUTER VISION FUNCTION CHEATSHEET

A quick-reference index of the functions, methods, properties and constants appearing in the supplied practice notebooks and the main notes. Arguments are shown in their commonly used form; optional parameters are omitted where they are not essential.

1. OpenCV image input, output and display

Function / property	Purpose and key syntax
cv2.imread(path, flag)	Read image from disk. Default is colour/BGR. Returns None if loading fails.
cv2.IMREAD_COLOR	Load a colour image (normally 3-channel BGR).
cv2.IMREAD_GRAYSCALE	Load a single-channel grayscale image.
cv2.imshow(title, img)	Display an image in a window.
cv2.waitKey(delay)	Wait for key input; delay is milliseconds; 0 waits indefinitely.
cv2.destroyAllWindows()	Close all OpenCV windows.
cv2.imwrite(path, img)	Save image to disk; returns success Boolean.
img.shape	Array dimensions: grayscale (H,W), colour (H,W,C).
img.dtype	Array element type, commonly uint8.
type(img)	Inspect Python object type; OpenCV image is usually numpy.ndarray.

2. Colour conversion, NumPy and Matplotlib

Function / property	Purpose and key syntax
cv2.cvtColor(img, code)	Convert colour spaces; e.g. BGR→GRAY or BGR→RGB.
cv2.COLOR_BGR2GRAY	Conversion code for BGR colour to grayscale.
cv2.COLOR_BGR2RGB	Conversion code for BGR to RGB display.
np.array(obj)	Convert image-like object to NumPy array.
np.zeros(shape, dtype)	Create an array filled with zeros; e.g. blank canvas.
array.copy()	Create an independent copy of an array.
array.shape / array.dtype	Read dimensions and element data type.
array[y, x] / array[:,:,c]	Access pixel or channel; NumPy order is row, column, channel.
plt.imshow(img)	Display an image using Matplotlib.
plt.imshow(img, cmap='gray')	Display a single-channel image in grayscale.
plt.axis('off') / plt.show()	Hide axes / render the Matplotlib figure.

3. Pillow (PIL) image methods

Function / method	Purpose and key syntax
Image.open(path)	Open an image file as a Pillow Image object.
img.rotate(angle)	Return a rotated image; angle is in degrees.
img.resize((W,H))	Return a resized image; size order is width, height.
img.crop((L,T,R,B))	Crop using left, top, right, bottom coordinates.
img.transpose(Image.FLIP_LEFT_RIGHT)	Return a left-right mirrored image.
img.filter(ImageFilter.BLUR)	Apply a Pillow image filter.
ImageFilter.BLUR	Built-in blur filter used in the notebook.
img.split()	Split image into separate channel images (e.g. R, G, B).
img.mode	Image mode, e.g. RGB, RGBA or L.

FUNCTION CHEATSHEET — CONTINUED

4. Geometric transformations and drawing

Function / constant	Purpose and key syntax
cv2.resize(img, (W,H))	Resize; dsize is (width,height), not (height,width).
cv2.resize(img, None, fx=sx, fy=sy)	Resize using horizontal and vertical scale factors.
cv2.INTER_AREA	Interpolation often suitable for shrinking.
cv2.INTER_LINEAR	Common interpolation method for resizing.
cv2.INTER_CUBIC	Higher-cost interpolation sometimes used for enlargement.
cv2.flip(img, 0)	Flip vertically (top ↔ bottom).
cv2.flip(img, 1)	Flip horizontally (left ↔ right).
cv2.flip(img, -1)	Flip both vertically and horizontally.
cv2.line(img, p1, p2, color, thickness)	Draw a line.
cv2.rectangle(img, p1, p2, color, thickness)	Draw rectangle from top-left to bottom-right.
cv2.circle(img, center, radius, color, thickness)	Draw circle; thickness=-1 fills it.
cv2.polylines(img, pts, isClosed, color, thickness)	Draw connected polygonal line segments.
cv2.putText(img, text, org, font, scale, color, thickness)	Draw text; org is bottom-left of text.
cv2.FONT_HERSHEY_SIMPLEX / PLAIN	Font-face constants used by putText().

5. Arithmetic and bitwise operations

Function	Purpose / formula
cv2.add(A,B)	Saturated pixel-wise addition for common uint8 images.
cv2.addWeighted(A, α, B, β, γ)	Weighted blend: dst = αA + βB + γ.
cv2.subtract(A,B)	Pixel-wise subtraction; uint8 negative results saturate at zero.
cv2.bitwise_and(A,B)	Bitwise AND; useful for masking.
cv2.bitwise_or(A,B)	Bitwise OR.
cv2.bitwise_xor(A,B)	Bitwise exclusive OR; set bits where inputs differ.
cv2.bitwise_not(A)	Invert bits; for uint8, output is 255 – input.

Important: XOR and subtraction are different operations and are not generally interchangeable. For bitwise functions, a mask can be supplied with the **mask=** keyword.

6. Filtering, thresholding and edge detection

Function / constant	Purpose and key syntax
cv2.GaussianBlur(img, (k,k), σ)	Gaussian smoothing; odd kernel dimensions; σ controls spread.
cv2.medianBlur(img, k)	Median filter; k must be odd and greater than 1.
cv2.filter2D(img, ddepth, kernel)	Apply a 2-D spatial filter; ddepth=-1 keeps input depth.
np.array([...])	Create the sharpening kernel matrix.
cv2.threshold(gray, T, maxVal, type)	Apply global threshold; returns (threshold_used, output_image).
cv2.THRESH_BINARY	Pixels > T become maxVal; others become 0.
cv2.THRESH_BINARY_INV	Inverse binary threshold.
cv2.THRESH_TRUNC	Values above T are clipped to T.
cv2.THRESH_TOZERO / _INV	Set one side of the threshold to zero.
cv2.Canny(gray, low, high)	Canny edge detector with lower and upper thresholds.

FUNCTION CHEATSHEET — CONTOURS & VIDEO

7. Contours and shape analysis

Function / constant	Purpose and key syntax
cv2.findContours(binary, mode, method)	Extract contours from a binary image. OpenCV 4 commonly returns (contours, hierarchy).
cv2.RETR_TREE	Retrieve contours and full nested parent-child hierarchy.
cv2.RETR_EXTERNAL	Retrieve only outermost contours.
cv2.CHAIN_APPROX_SIMPLE	Compress contour representation by retaining key points.
cv2.drawContours(img, contours, idx, color, thickness)	Draw contours; idx=-1 draws all contours.
cv2.contourArea(contour)	Calculate enclosed contour area.
cv2.arcLength(contour, closed)	Calculate contour perimeter; closed=True for closed boundary.
cv2.approxPolyDP(contour, epsilon, closed)	Simplify contour using Douglas–Peucker approximation.
len(approx)	Count approximated vertices; used in the notebook's shape heuristic.
cv2.putText(...)	Label the detected shape on the image.

8. Webcam and video capture

Function / method	Purpose and key syntax
cv2.VideoCapture(0)	Open default webcam; use a filename for a video file.
cap.isOpened()	Check whether camera/video source opened.
cap.read()	Read next frame; returns (ret, frame).
cap.release()	Release camera/video resource.
cv2.waitKey(1)	Allow window refresh and check keyboard input in a live loop.
ord('q')	Get character code used to detect q key.
try / finally	Python control structure used to ensure camera release.

9. Haar-cascade face detection

Function / method	Purpose and key syntax
cv2.CascadeClassifier(xml_path)	Load pretrained cascade model from XML.
cascade.empty()	Check whether cascade failed to load; True means empty.
cascade.detectMultiScale(gray, scaleFactor, minNeighbors)	Detect objects at multiple image scales; returns rectangles.
scaleFactor=1.1	Scale step between successive image sizes; smaller values scan more scales.
minNeighbors=5	Grouping threshold; higher values usually make detections more conservative.
(x,y,w,h)	Detection rectangle: top-left coordinate, width and height.

10. Fast parameter reminders

Parameter	Remember
(W,H)	OpenCV resize dsize and Pillow resize use width first, height second.
(x,y) vs img[y,x]	Drawing coordinates are (x,y); NumPy pixel indexing is [row=y, column=x].
Colour tuples	OpenCV drawing and loaded colour images use BGR order.
Gaussian kernel	Use odd dimensions such as (3,3), (5,5), (7,7).
Median ksize	Odd integer greater than 1, e.g. 3 or 5.
Threshold T	Controls intensity split; maxVal is commonly 255 for 8-bit images.
Canny low/high	Two thresholds; high threshold should be greater than low threshold.
approxPolyDP epsilon	Larger epsilon generally gives fewer polygon vertices.
VideoCapture	Always release the capture and close windows after the loop.

Interview tip: Do not memorise only function names. For each function, know its input, output, one important parameter, the problem it solves, and one limitation.

COMPUTER VISION NOTES

Supplementary update: topics cross-checked against the supplied handbook

Purpose: This addendum expands the existing notes and function cheatsheet. It distinguishes concepts present in the supplied reference from techniques confirmed in the original practical notebooks.

Practical evidence boundary: The original notebooks demonstrate image I/O and manipulation, BGR/RGB conversion, arithmetic and bitwise operations, Gaussian and median blur, kernel sharpening, binary inverse thresholding, Canny, contour analysis, webcam capture, and Haar face/eye/smile detection. The additional sections below are study/reference material unless separately implemented.

A. Topics to add to the theory notes

1. HSV colour space

HSV means Hue, Saturation and Value. Hue describes the colour family, saturation describes colourfulness or purity, and value describes brightness. HSV is useful for colour-based segmentation because hue and saturation can help distinguish an object from its background when brightness varies. It does not make segmentation automatically illumination-proof.

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lower = np.array([H_min, S_min, V_min])
upper = np.array([H_max, S_max, V_max])
mask = cv2.inRange(hsv, lower, upper)
```

Function: `cv2.inRange(src, lowerb, upperb)` creates a binary mask: 255 where every channel lies within the inclusive bounds, otherwise 0. HSV range values must match OpenCV's channel conventions and data type.

2. Resizing, interpolation and cropping

Resizing changes image dimensions. Interpolation estimates values at newly sampled pixel locations. Common choices: INTER_NEAREST (nearest sample; useful for label masks), INTER_LINEAR (bilinear; common general-purpose choice), INTER_CUBIC (bicubic; often used for enlargement), and INTER_AREA (area relation; often suitable for shrinking).

```
small = cv2.resize(image, (320, 240), interpolation=cv2.INTER_AREA)
large = cv2.resize(image, (640, 480), interpolation=cv2.INTER_CUBIC)
roi = image[y1:y2, x1:x2]
```

OpenCV's resize size is (width, height), while NumPy image indexing is [row, column] = [y, x]. Cropping by slicing returns a view in common NumPy cases; use .copy() when an independent crop is needed.

3. Additional smoothing: averaging and bilateral filtering

Averaging blur replaces a pixel with the arithmetic mean of a neighbourhood. It is a linear low-pass filter and can soften edges. Bilateral filtering combines spatial proximity and intensity similarity, so it can smooth relatively uniform regions while preserving many edges better than a simple blur.

```
average = cv2.blur(image, (5, 5))
bilateral = cv2.bilateralFilter(image, d=9,
                                sigmaColor=75, sigmaSpace=75)
```

These are useful reference techniques in the supplied handbook; they were not identified as completed practicals in the original notebooks.

4. Thresholding beyond the practical binary-inverse example

Global thresholding uses one threshold for the whole image. Adaptive thresholding computes a local threshold for each neighbourhood and can help when illumination is uneven. Otsu's method automatically selects a threshold by maximising between-class variance (equivalently minimising within-class variance for a two-class split).

```
# Adaptive thresholding: input must be single-channel 8-bit image
adaptive = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2
)

# Otsu thresholding
threshold_used, otsu = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Method	How output is decided	Useful when
Binary	Above threshold -> max value; otherwise 0	Foreground is brighter
Binary inverse	Above threshold -> 0; otherwise max value	Foreground is darker

Method	How output is decided	Useful when
Truncate	Values above threshold are clipped to threshold	Limiting high intensities
To zero	Keep values above threshold; set others to 0	Suppressing darker pixels
To zero inverse	Keep values at/below threshold; set others to 0	Suppressing brighter pixels
Adaptive	Local neighbourhood threshold	Uneven illumination
Otsu	Automatically estimates one global threshold	Often bimodal grayscale histograms

5. Sobel and Laplacian edge operators

Sobel estimates first-order intensity derivatives. The x derivative responds strongly to vertical boundaries; the y derivative responds strongly to horizontal boundaries. The gradient magnitude combines both responses. Laplacian uses a second derivative and responds to rapid intensity changes in multiple directions, but it is sensitive to noise.

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
magnitude = cv2.magnitude(sobel_x, sobel_y)
sobel_display = cv2.convertScaleAbs(magnitude)

lap = cv2.Laplacian(gray, cv2.CV_64F)
lap_display = cv2.convertScaleAbs(lap)
```

Gradient magnitude: G = sqrt(Gx^2 + Gy^2). Gradient direction is best represented computationally as atan2(Gy, Gx), which handles quadrants more reliably than atan(Gy/Gx). Sobel and Laplacian are additional reference topics, not original notebook practicals.

6. Contour geometry and hierarchy

Function / concept	Meaning / return
cv2.boundingRect(c)	Upright bounding rectangle (x, y, width, height).
cv2.minEnclosingCircle(c)	Smallest enclosing circle: ((center_x, center_y), radius).
cv2.moments(c)	Dictionary of spatial/central moments; centroid uses m10/m00 and m01/m00 when m00 != 0.
cv2.contourArea(c)	Area enclosed by a contour, in squared pixel units.
cv2.arcLength(c, True)	Closed contour perimeter; False measures an open curve.
cv2.approxPolyDP(c, epsilon, True)	Polygonal approximation; larger epsilon usually simplifies more.

Contour retrieval modes: RETR_EXTERNAL returns only outermost contours; RETR_LIST returns all contours without parent/child hierarchy; RETR_CCOMP organises contours into a two-level hierarchy; RETR_TREE reconstructs the full nesting hierarchy. CHAIN_APPROX_NONE stores all boundary points; CHAIN_APPROX_SIMPLE compresses straight horizontal, vertical and diagonal runs.

Shoelace polygon-area formula: A = 1/2 * |sum(x_i*y_(i+1) - x_(i+1)*y_i)|, with the final vertex connected to the first. Contour moments provide a centroid: Cx = M10/M00 and Cy = M01/M00, provided M00 is nonzero.

7. Haar Cascade: fuller theory

Haar-like features compare sums of pixel intensities in adjacent rectangular regions. The integral image (summed-area table) makes rectangle sums efficient: a rectangle sum can be computed from four integral-image corner values, conventionally S = D - B - C + A. AdaBoost selects useful features and combines weak classifiers. A cascade applies classifier stages sequentially: easy negative windows are rejected early; candidate windows that pass continue to later stages.

detectMultiScale parameters: scaleFactor controls the image-pyramid scale step (for example, 1.1 is about a 10% step); minNeighbors controls how many neighbouring candidate rectangles are needed to retain a detection; minSize sets the minimum candidate dimensions. Higher minNeighbors usually reduces false positives but may also reject true detections.

Detection is not recognition. A face detector locates face-like regions; it does not identify the person's identity.

8. Region of Interest (ROI), eyes and smile

An ROI is a selected subregion processed separately. After detecting a face at (x, y, w, h), the grayscale face crop can be extracted as gray[y:y+h, x:x+w]. Eye or smile detection within this face ROI reduces irrelevant search area and can reduce false detections. ROI coordinates are local to the cropped region; add the face's x and y offsets if drawing those detections on the full image.

```
face_roi_gray = gray[y:y+h, x:x+w]
```

```
eyes = eye_cascade.detectMultiScale(face_roi_gray, 1.1, 5)

for (ex, ey, ew, eh) in eyes:
    # Convert ROI-local coordinates to full-image coordinates
    cv2.rectangle(image, (x+ex, y+ey),
                  (x+ex+ew, y+ey+eh), (255, 0, 0), 2)
```

B. Added OpenCV function cheatsheet

These functions are present in the supplied handbook but were missing from, or not fully explained in, the earlier cheatsheet. The purpose column is a quick recall aid; refer to the relevant chapter for theory and limitations.

Function / constant	Purpose / key reminder
cv2.COLOR_BGR2HSV	BGR-to-HSV conversion code for cv2.cvtColor.
cv2.inRange()	Create binary mask for values within lower/upper channel bounds.
cv2.INTER_NEAREST	Nearest-neighbour resize interpolation; useful for categorical masks.
cv2.INTER_LINEAR	Bilinear interpolation; common resize default.
cv2.INTER_CUBIC	Bicubic interpolation; often used for enlargement.
cv2.INTER_AREA	Area-based interpolation; often useful for reduction.
cv2.blur()	Normalised box/averaging blur.
cv2.bilateralFilter()	Edge-preserving smoothing using spatial and intensity similarity.
cv2.adaptiveThreshold()	Local thresholding for varying illumination.
cv2.ADAPTIVE_THRESH_MEAN_C	Local arithmetic-mean adaptive threshold method.
cv2.ADAPTIVE_THRESH_GAUSSIAN_C	Gaussian-weighted local adaptive threshold method.
cv2.THRESH_TRUNC	Truncate pixel values above threshold.
cv2.THRESH_TOZERO	Set values at/below threshold to zero.
cv2.THRESH_TOZERO_INV	Set values above threshold to zero.
cv2.THRESH_OTSU	Flag to combine with threshold type for automatic Otsu threshold.
cv2.Sobel()	First-order x/y derivatives; specify dx, dy and output depth.
cv2.Laplacian()	Second-order derivative edge response.
cv2.magnitude()	Calculate sqrt(x^2 + y^2) from two gradient arrays.
cv2.convertScaleAbs()	Convert/scale absolute values to 8-bit displayable output.
cv2.boundingRect()	Get upright contour bounding box.
cv2.minEnclosingCircle()	Get smallest circle enclosing a contour.
cv2.moments()	Calculate contour/image moments.
cv2.RETR_LIST	Retrieve all contours without hierarchy relationships.
cv2.RETR_CCOMP	Retrieve contours in a two-level hierarchy.
cv2.CHAIN_APPROX_NONE	Store all contour boundary points.
cv2.inRange()	Threshold each channel between lower and upper bounds.

C. Consolidated code patterns for the added topics

1. HSV colour mask

```
import cv2
import numpy as np

image = cv2.imread("image.jpg")
if image is None:
    raise FileNotFoundError("Could not read image.jpg")

hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lower = np.array([35, 50, 50], dtype=np.uint8)
upper = np.array([85, 255, 255], dtype=np.uint8)
mask = cv2.inRange(hsv, lower, upper)
result = cv2.bitwise_and(image, image, mask=mask)
```

2. Adaptive and Otsu thresholding

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

adaptive = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
```

```
    cv2.THRESH_BINARY, 11, 2
)
otsu_t, otsu = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

3. Sobel and Laplacian

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
grad = cv2.convertScaleAbs(cv2.magnitude(gx, gy))

lap = cv2.Laplacian(gray, cv2.CV_64F)
lap = cv2.convertScaleAbs(lap)
```

4. Contour centroid and bounding geometry

```
contours, hierarchy = cv2.findContours(
    binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE
)
for contour in contours:
    x, y, w, h = cv2.boundingRect(contour)
    (cx, cy), radius = cv2.minEnclosingCircle(contour)
    M = cv2.moments(contour)
    if M["m00"] != 0:
        centroid = (int(M["m10"]/M["m00"]),
                    int(M["m01"]/M["m00"]))
```

5. Camera loop safety pattern

```
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open camera")

try:
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        cv2.imshow("Camera", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
finally:
    cap.release()
    cv2.destroyAllWindows()
```

D. Interview comparisons and correction notes

Question	Strong concise answer
HSV vs BGR/RGB?	BGR/RGB encode colour channel values in different orders; HSV separates hue, saturation and value and can be useful for colo
Global vs adaptive threshold?	Global uses one threshold for the image; adaptive computes local thresholds and handles uneven illumination better.
Adaptive vs Otsu?	Adaptive is spatially local; Otsu selects one automatic global threshold based on histogram class separation.
Sobel vs Laplacian?	Sobel estimates first derivatives in selected directions; Laplacian uses a second derivative and is more noise-sensitive.
Canny vs Sobel?	Sobel gives gradient responses; Canny adds smoothing, non-maximum suppression, double thresholding and hysteresis.
Bounding rectangle vs enclosing circle?	The rectangle is axis-aligned and returns x,y,w,h; the circle returns a centre and radius.
Contour area vs perimeter?	Area measures enclosed region; arcLength measures boundary length.
Face detection vs recognition?	Detection locates faces; recognition attempts to determine or verify identity.
Why ROI?	It restricts processing to a relevant region, reducing search area and irrelevant detections.

Important implementation cautions

1. OpenCV image arrays normally use BGR channel order; Matplotlib normally expects RGB for colour display.

2. OpenCV drawing coordinates are (x, y), while NumPy indexing is image[y, x].

3. For ROI-based eye detection, detections are relative to the ROI. Add the face offset when drawing on the original frame.

4. cv2.findContours return signatures differ across OpenCV versions. Modern OpenCV 4 commonly returns contours, hierarchy; older examples may show an additional modified-image return value.

5. Otsu and adaptive thresholding, HSV masking, averaging/bilateral blur, Sobel, Laplacian, and the extra contour geometry methods are added as theory/code references from the supplied handbook; do not describe them as completed original notebook experiments unless you implement them.

6. A simple polygon-vertex-count rule (triangle/square/pentagon) is a teaching heuristic. Real shape classification needs robust handling of contour quality, rotation, perspective, scale and noise.

E. Final expanded topic checklist

Core practicals already in original notebooks	Extra study/reference topics added from supplied handbook
Image read/write/display; dimensions and NumPy arrays	HSV and colour masking with inRange
BGR/RGB/grayscale conversion	Interpolation methods and resize choices
Resize, crop, flip, rotate, Pillow transforms	Averaging and bilateral filtering
Gaussian blur; median blur	Adaptive thresholding and Otsu's method
Sharpening with filter2D kernel	Sobel and Laplacian edge operators
Arithmetic and bitwise operations	Bounding rectangle, enclosing circle and moments
Binary inverse thresholding	Contour retrieval modes and full hierarchy
Canny edge detection	Haar integral image, AdaBoost and cascade-stage explanation
Contours, area, perimeter, approximation and shape heuristic	ROI-based eye/smile detection code pattern
Webcam capture loop	Expanded interview comparisons and consolidated code patterns
Haar face/eye/smile cascade workflow	Additional API/function quick reference

Study recommendation: First master the original practicals and be able to explain each input, output and parameter. Then study the added topics as theory and practise their short code patterns. This is a stronger and more honest preparation strategy for both college viva and entry-level interviews.